MORA · BACKEND · CRITÉRIO DE AVALIAÇÃO

Serviços do backend corretamente documentados

Como o projeto se adequou ao critério · Peso 1.0 · Outubro de 2026 · PRs #32 e #33 (mesclado na `develop`)

> **Rubrica.** a) Todos os serviços documentados numa página, com sumário e descrição preenchidos; b) exemplos de payload com dados reais; c) README completo.

7	342	0	130
serviços no portal	operações documentadas	operações sem sumário, descrição ou exemplo	asserções do teste de API, 0 falhas

1. Onde estávamos

Serviço	Situação antes
comunicacao, portaria	Swagger no ar, mas sem nenhuma descrição nem exemplo
auth-api, financeiro, gestao-geral	Sem OpenAPI
plan	Swagger respondia 500 (springdoc 2.3.0 no Spring Boot 3.5) e o serviço não estava no gateway
meeting	`@Operation` em parte das rotas

Cada serviço tinha, no máximo, a própria página, em portas diferentes, e não havia exemplo de payload em lugar nenhum.

2. O que fizemos

a) Uma página com todos os serviços

O portal fica em `http://localhost:8087/docs`, pelo API Gateway (Traefik), servido pelo gestao-geral.

- **No topo, o sumário:** uma linha por serviço com o que ele faz, responsável, banco, número de rotas e situação (no ar ou fora do ar), lidos ao vivo da especificação de cada um.
- **Abaixo, o Swagger UI** com um seletor dos 7 serviços. O *Try it out* passa pelo gateway; testado com `GET /api/portaria/blocos`, que respondeu 200.
- **Toda operação** tem sumário, descrição, parâmetros e respostas de erro, agrupada por tags com descrição.

Serviço	Operações	Como a especificação é produzida
auth-api	63	Escrita à mão (`docs/openapi.js`) e conferida contra o router do Express por teste
financeiro	53	Idem
gestao-geral	10	Idem; também serve o portal
comunicacao	47	Anotações springdoc (`@Tag`, `@Operation`, `@ApiResponse`, `@Schema`)
portaria	151	Anotações, com os exemplos num arquivo (`openapi/exemplos.json`) injetado por um `OpenApiCustomizer`
plan	18	Anotações, com exemplos nos campos dos DTOs
meeting	anotado	Aparece no sumário; a especificação carrega quando o serviço está no ar (ele depende das credenciais do Google)

b) Exemplos com dados reais

Todo corpo de requisição e toda resposta de sucesso têm exemplo. Os exemplos usam os **dados do seed de desenvolvimento**: Residencial Parque Verde, Bloco A, apartamento 101, `sindico.verde@mora.test`, valores em centavos. Os ids e e-mails existem no banco, então o *Try it out* funciona com o próprio exemplo.

Regras seguidas: nenhuma senha, token real ou dado de pessoa real nos exemplos. Token aparece truncado (`eyJhbGciOi...`) e CPF é fictício e válido.

c) README

O README ganhou a seção "Documentação da API", com o endereço do portal, o endereço da especificação de cada serviço e a relação entre os exemplos e o seed.

Correções que o critério exigiu

- **plan:** springdoc de 2.3.0 para 2.8.9 (o 500 sumiu), rotas do Swagger liberadas no filtro de autenticação, registro no Consul ligado e `/api/assinaturas` nas regras do Traefik.
- **comunicacao e portaria:** `forward-headers-strategy`, para o Swagger montar o endereço com o prefixo do gateway e o *Try it out* funcionar por ele.

3. Como se garante que continua assim

Teste que quebra se uma rota ficar sem documentação. Nos três serviços Node, um teste lê as rotas registradas no Express e exige, para cada uma, a operação na especificação, com sumário, descrição e exemplo quando houver corpo.

Conferência ponta a ponta pelo gateway. O teste de API (Postman + Newman, pasta `tests-api`) tem uma pasta "00 Documentação": abre o portal e confere cada especificação, operação por operação. Última execução: 57 requisições, 130 asserções, 0 falhas.

4. Como verificar

```
docker compose -f docker/docker-compose.yml up -d --build
# abrir no navegador
http://localhost:8087/docs

# teste de API (com o seed aplicado)
npm run seed
cd tests-api && npm test
```

5. Situação

Item da rubrica	Situação
a) Página única com sumário e descrição	Atendido
b) Exemplos de payload com dados reais	Atendido
c) README completo	Atendido

Ressalva: o meeting-service só aparece com a especificação carregada quando está no ar, e para subir ele precisa do `credentials.json` e do token do Google (arquivos fora do Git por conterem segredo). Sem eles, o portal mostra o serviço como "fora do ar".